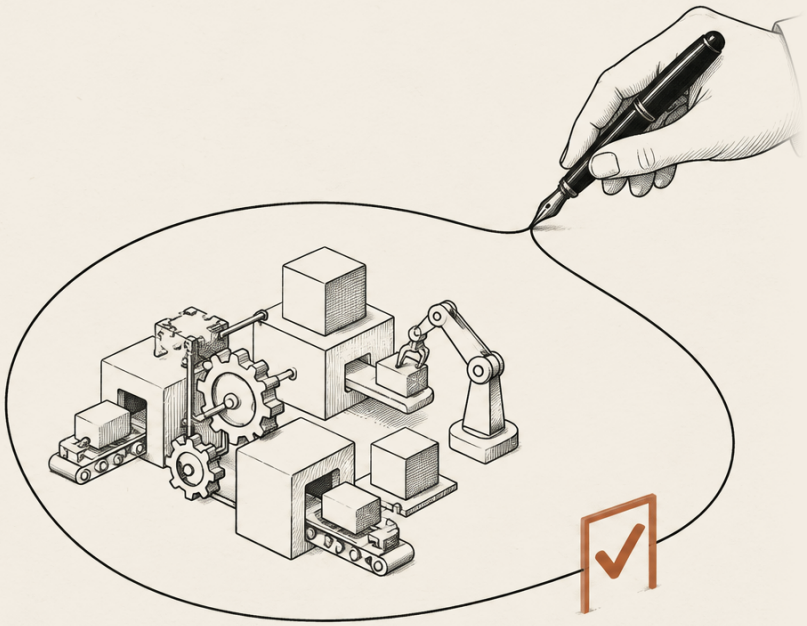
Vibe Coding in Practice

Organize work with prompts
Judge results by evidence

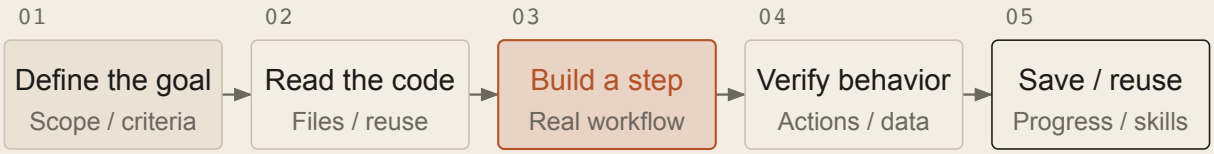


Bingjie Zhu

October 2026

From an Idea to a Usable Result

Describe a goal in natural language. Let a coding agent read the project, edit files, run checks and improve the result. The agent repeatedly uses tools and chooses its next step from what it observes.



Parallelize independent work; return to the relevant step when problems appear.

One workflow, with methods added as needed

Clarify the goal, research existing solutions, then ask AI to implement, check and revise. Keep requirements, evidence and progress in project files; add collaboration and automation when useful. Understand what each method solves and when to use it. AI can help look up the tool-specific steps.

Current situation	What to do
The idea is still vague	Clarify scope and acceptance criteria; plan a minimal usable version
Starting or changing a project	Read the project, reuse existing work, make a small change and run it
The result is wrong	Check the real workflow, keep error evidence and diagnose before fixing
Several independent tasks	Investigate in parallel; coordinate edits and review important results independently
Work is interrupted or repeated	Save progress, capture reusable methods, then consider batch execution

How to use the shaded prompt boxes

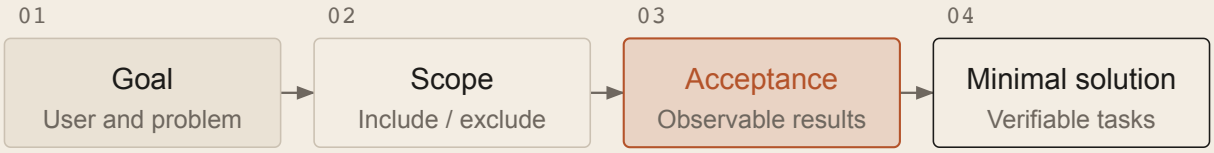
1. **Find and copy.** Each large shaded box in a chapter contains a prompt. Choose one for your task and copy all the text below its title.
2. **Replace the brackets.** Fill in placeholders such as “[goal]” with your requirements, for example: “Add a to-do item and keep it after a refresh.”
3. **Paste and send.** Open the project in Codex or Claude Code, send the completed prompt in the chat, then run and check the result as directed.

Where human judgment matters

People decide goals, trade-offs and whether to accept the result; AI compares options, executes and provides evidence. Here, *taste* means understanding users and setting priorities. A demo may emphasize presentation; a daily-use tool may prioritize reliability. Meet essential requirements before balancing experience, cost and speed. Express preferences through scenarios, examples and observable criteria, and check whether those criteria fit the task.

CHAPTER 1

Turn an Idea into an Actionable Task



Settle important choices before implementation.

Describe the result first

1. **State the goal and its basis.** Say who will use it and what they need to do. Include existing materials or references you approve of. Describe the outcome without prescribing every technical step.
2. **Set the scope of version one.** Separate what must work now from what can wait. If the idea is vague, let AI ask one key question at a time to clarify scenarios, boundaries and trade-offs. Do not repeatedly reconfirm settled requirements.
3. **Agree on acceptance criteria.** Replace "usable, complete, professional" with observable behavior: which operations must succeed, how failures should be handled and what must stay unchanged. Check for missing requirements before planning implementation.

Clarify Requirements and Propose a Minimal Solution

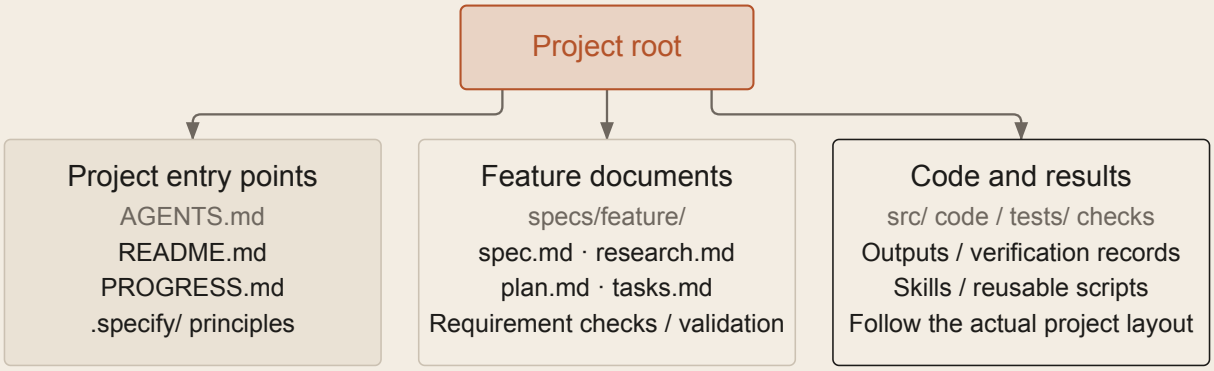
I want [users] to accomplish [goal] in [scenario]. The project and reference materials are at [location; say if none exist]. This version must include [scope] and exclude [out of scope]. You may edit [allowed scope].
Check the current state and preserve the original requirements. In the existing requirements document or the current feature's spec.md, record the goal, key user actions, acceptance criteria, behavior to preserve and open questions. Verify what you can; ask about unclear requirements one at a time, offer recommendations and label unconfirmed assumptions.
Propose a minimal solution and small tasks with a check for each. This round is for requirements and an initial outline, not code changes; defer technical choices until research. Follow existing documents or Spec Kit steps. Report inaccessible materials; if you cannot write files, return a draft without claiming it was saved.

Keep important choices with people; leave room for implementation

A specification (spec) states goals, behavior, constraints and acceptance criteria; a plan explains the proposed implementation. Discuss the idea first, then write the spec. People review goals, trade-offs and acceptance; research informs the technical design. Clear criteria help execution but do not guarantee correct interpretation or implementation. A small, well-defined change may need only a short instruction rather than a long plan.

CHAPTER 2

Connect Project Files and Responsibilities



Typical filenames; Spec Kit uses the currently selected feature directory.

Files preserve requirements beyond the chat. A trailing / in the diagram means a folder; `.md` is readable Markdown text. `README.md` explains how to run and check the project. `AGENTS.md` is a rules format supported by agents such as Codex. Direct loading in Claude Code depends on version and configuration; `CLAUDE.md` can also import it with `@AGENTS.md`. `.specify/` stores project principles and workflow support; feature requirements belong in the active feature directory.

File	Questions it answers
`spec.md`	What to build, what to exclude and what counts as done; preserve original wording or a traceable link
`research.md`	What was investigated, sources, alternatives, rationale and remaining unknowns; link experiment records
`plan.md` / `tasks.md`	Implementation approach; ordered tasks, status and checks
`PROGRESS.md`	Actual progress across sessions, unfinished work and where to resume
Verification records	What ran, against which version, results and failure evidence

Organize Project Files and Reading Entry Points

Project: [path]. Task: [goal]. Documents you may update: [scope]. Locate the existing rules, requirements, research, plan, tasks, progress and evidence. Add files only when missing and useful. Use the active Spec Kit feature directory rather than creating parallel copies. Report unreadable paths; return a draft if you cannot write.

Keep rules limited to project-specific conventions, edit scope, check commands and document pointers. Use a short index linking research and experiments to their originals. Preserve valid content; do not move or delete originals. List updated paths and when to maintain them. Refresh rules as the tool requires and check their loaded sources; if loading is not observable, mark it unverified. Creating a file does not prove it is active.

Use an index to find information; keep originals as evidence. Context compaction itself does not rewrite project files on disk. Save key information beforehand and make it accessible in later sessions. A file's existence does not mean the agent has read it. On resuming, reread the current spec, progress and relevant evidence. Update plans and tasks when requirements change, then record results. Small tasks can share one short record.

CHAPTER 3

Understand the Project Before Changing It



Use a short index to current files; read details as needed.

Research existing solutions before choosing an architecture

Open the right project. Inspect existing implementations and similar projects before choosing reuse, adaptation or a new build. Compare use cases, costs and limits using official documentation, original research and examples. Discuss module responsibilities, data flow and interfaces (agreements for exchanging data). Ask why the design fits and when it would not; build on existing work.

Protect data before starting

1. **Use a test environment.** Work with data copies and test accounts; restrict file and network access. Isolate production when experimenting with broader permissions. Before deleting, publishing or changing production data, establish authorization, backups and recovery steps.
2. **Protect secrets.** Prefer a secrets manager. Restrict access to local `.env` files and never commit them or paste secrets into chat or client-side code. `.gitignore` does not untrack files, stop AI reads or erase history. Revoke and replace leaked credentials.
3. **Inspect unfamiliar content.** Instructions in a page or file that divert AI from its task are prompt injection. Avoid combining access to private data with outbound or deletion permissions when reading untrusted content. Check the origins of third-party skills, scripts and packages.

Read the Project and Identify the Change

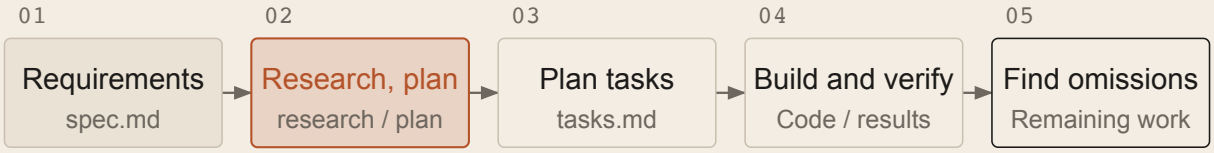
Goal: [task]. Materials: [location]. Research file you may update: [path]. Check the version, existing changes, rules and runtime requirements. Do not change the program this round; validate only in a test environment. Report unreadable materials; return a draft if you cannot save records. Find existing implementations and comparable solutions. Compare reuse, adaptation and a new build. Verify package names, official sources and licenses. If offline, label external claims unverified; do not invent sources. Run minimal experiments on architecture-relevant unknowns and explain modules, data flow and dependencies.
Record questions, sources and check dates, observations, alternatives, rationale and unknowns. Separate established facts, inferences and hypotheses; say what new evidence would change the choice. Report actual edits and unverified items. Preserve original materials and verification evidence.

Make research inform a decision

Context is the information AI can currently access; provide it as needed. Research should answer what can be reused, why it fits and what still needs validation, then inform the plan. A file's presence does not prove it was read. A tool running locally may still send information to a cloud model.

CHAPTER 4

Connect Specs and Execution with Spec Kit



When evidence changes the plan, update its files and affected tasks first.

A spec describes requirements. Spec Kit is an optional open-source tool for organizing specs, plans and tasks. Understand the method before adopting the tool; a short note can be enough for a small change.

Steps and files: check your installed version

Step	Purpose and what to check
`constitution`	Set principles in `.specify/memory/constitution.md`
`specify / clarify`	Define requirements, resolve ambiguity and maintain `spec.md`
`plan`	Record research in `research.md` and the approach in `plan.md`
`checklist`	Check requirements quality as needed; passing is not implementation
`tasks / analyze`	Create `tasks.md`; check consistency with requirements and plans
`implement`	Execute tasks, run checks and keep results
`converge`	Find omissions and add remaining work; still verify actual behavior

`plan` also produces `data-model.md` (data structure) and `quickstart.md` (validation steps), with `contracts/` for external interfaces where applicable.

Current default skill invocation: Codex uses `$speckit-specify`; Claude Code and Copilot use `/speckit-specify`. The active feature is selected by `.specify/feature.json` or runtime settings, not automatically by switching Git branches.

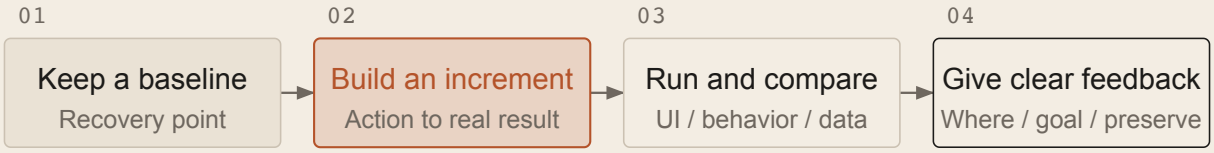
Advance the Current Feature with Spec Kit

Project: [path]. Basis: [requirements and records]. Step for this round: [step]. Allowed edits: [scope]. Check the installed version, invocation, active feature and prerequisites. If installation or inputs are missing, propose how to supply them; do not install or skip ahead on your own.
Preserve existing files. Clarify ambiguity and investigate unknowns. Include required automated tests in the spec and tasks; map each task to requirements and checks.
If only the user can invoke the step, provide the command and wait for its result. Report actual outputs, checks, conflicts and the next step. Do not claim an uninvoked step ran. Surface contradictory requirements rather than choosing silently.

When unsure, have AI check the official instructions and actual invocation. People review goals and acceptance; AI implements tasks. Update the plan and checks when requirements change so new prompts do not overwrite prior agreements. Complete documentation does not prove a finished feature.

CHAPTER 5

Build in Small Steps and Give Specific Feedback



Focus each round on related changes; check before continuing.

Complete one real user workflow at a time

1. **Keep a recoverable version.** Git records and compares versions. Protect existing changes, then use a branch (a separate line of development) for this round. After a checked increment, make a local commit explaining the change. A commit saves locally; a push uploads to a remote repository. Tool undo history is not a substitute for version control.
2. **Connect one real path first.** Let a user reach an actual result from the first action, and persist anything that must be saved before adding features. For a UI, preview the layout before connecting real data; a finished preview is not a working feature.
3. **Give feedback on what you see.** Provide screenshots or references. State the location, current behavior, desired behavior and what must stay unchanged. Focus each round on related issues, then reopen and check.

Implement One Verifiable Increment

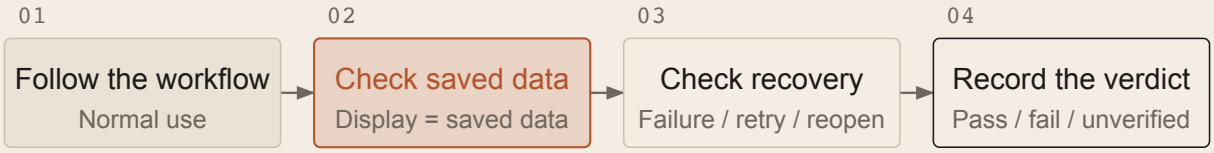
Project and requirements: [location; state requirements if no document exists]. Complete [feature scope] using [key checkpoints]. You may edit [scope]. Preserve [behavior and style]. References: [files or screenshots; say if none].
Inspect existing changes and keep a recoverable baseline using version history or a backup; exclude unrelated work. Implement, check and fix against [runnable acceptance checks] within [budget]. Report missing key inputs. Pause affected work for unauthorized actions, important trade-offs or repeated failures with no new lead. Connect the entry point, processing, persistence and feedback. Label mock data, unfinished features and hard-coded answers.
Explain key changes and data destinations, with file locations. Run the project and provide an accessible address or path, check steps and actual results. For UI changes, click, type, refresh and compare screenshots; without tools, give manual steps and mark those checks unverified. Save this round separately; commit locally only if authorized, without pushing. Update tasks and progress. Use [location, current behavior, desired behavior, things to preserve] for the next change. Avoid unrelated rewrites; explain trade-offs before changing agreed goals.

Each round should leave something worth keeping

The usable version saved before editing is the baseline. Keep its version, run instructions and check results before the next round; use the same criteria to compare old and new. Small changes help locate improvements and regressions, avoiding a rewrite that loses accepted behavior. Try the key workflow yourself before giving feedback. If the result is no better, restore the baseline or reduce the change.

CHAPTER 6

Judge Completion Through Actual Use



No runtime error is not proof that the original goal was met.

Check what users will actually experience

Area	Expected evidence
Normal use	Complete the goal from the real entry point; displayed and saved data agree
Errors and repetition	Missing input, failed operations and retries are handled without unintended duplicate effects
Reopening and recovery	Required data survives refreshes, restarts or interruptions; work can resume as agreed
Permissions and existing features	Unauthorized accounts cannot access or change data; related behavior still works

Run Acceptance Checks Against Requirements

Check [project location and version] against [requirements and original materials]. Approved test environment and data: [scope]. List expected outcomes first; clarify important ambiguity. Do not treat current program output as the correct answer. This round is for acceptance, not changing implementation or weakening criteria.

Follow the complete real workflow. Compare displayed results with saved records. Based on risk, check failures, repetition, reopening, recovery and existing features. Add required tests only within scope; a regression check should expose the original error before passing after the fix. For accounts or public endpoints, check unauthorized access, exposed secrets and external input being executed as commands, within authorization.

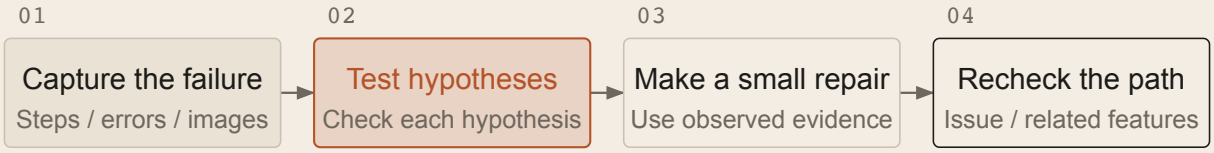
For each requirement, report steps, expected and observed results, evidence, and pass, fail or unverified status. Identify mock data, substitute services and environment limits. Respect authorization for checks with external effects. If unable to run, give manual steps and mark gaps unverified. No error message is not proof of completion. Do not delete, skip or relax checks to obtain a pass; report conflicts between requirements and checks.

Tie completion claims to verification results

An end-to-end check follows the user's workflow from start to finish. Establish that a check can detect a missing or wrong result before relying on its passing after a fix. Compare reports with original requirements and operational evidence; missing evidence remains unverified. AI-written code and tests can share the same misunderstanding. Working features are not necessarily secure: before real use, also inspect permissions, secrets in code and logs, and recovery procedures. Have an experienced person review important issues.

CHAPTER 7

Diagnose Before Changing Code



Each attempt should add evidence; avoid unsupported rewrites.

Turn a failure into useful evidence

1. **Preserve the full symptom.** Record steps, expected and actual outcomes, exact error messages with secrets removed, screenshots and recent changes. “It does not work” is not enough.
2. **Distinguish possible causes.** Have AI check paths, runtime settings, permissions and data, then propose causes and checks that distinguish them. Each attempt should add evidence.
3. **Fix only what the evidence supports.** Repeat the original action after a fix, then check related behavior. For intermittent failures, record attempts and failures; temporary disappearance does not prove the cause was resolved.

Investigate a Specific Failure

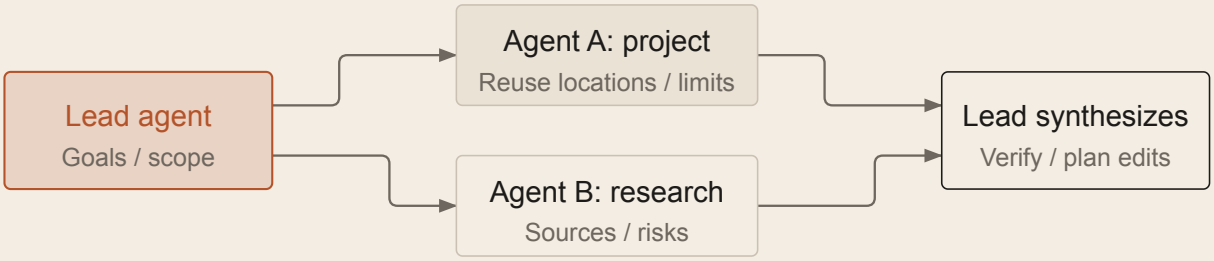
Problem: [steps, expected outcome, actual outcome]. Project and error records: [location]. Recent changes: [details; say if unknown]. Allowed repairs: [scope]. Investigation budget: [time or attempts].
Check the current version, runtime settings, permissions and files before rewriting. Reproduce in an approved test environment. If intermittent, record conditions, attempts and failures. Report missing inputs or tools; do not turn guesses into conclusions.
List possible causes and their predicted observations. Run minimal checks to find the earliest divergence. Preserve key experiments: hypothesis, version, conditions, action, result and evidence; keep unsuccessful paths too. Make a small repair only with supporting evidence and within scope. Where automated reproduction is possible, retain a check that fails before the fix and passes after it, then check related features. If an approach repeatedly fails or the budget is nearly spent, save facts and unknowns and consider an independent review. Return records if you cannot write them. Exclude sensitive information.

Failure records are project assets

Reproducing means triggering the same error under the same conditions; a regression occurs when a change breaks existing behavior. Preserve failing examples, ruled-out causes and checks after a repair. Consult them when a similar issue returns to avoid repeating ineffective attempts. Earlier conclusions apply to their original conditions; a new version or environment may justify retesting. A regression check is more useful than a note saying “be careful next time.”

CHAPTER 8

Split Independent Work Across Agents



Multiple agents can collaborate. This chapter uses a lead agent to assign and synthesize work: the main session owns the goal, while subagents handle bounded tasks. Each can read extensive material and return conclusions with evidence pointers, reducing clutter in the main session. Parallelize independent research first; coordinate tightly coupled edits.

Start with two read-only assistants

1. **Request real subagents in the main session.** Confirm support and inspect task records. "Read-only" instructions are not access controls. Codex subagents inherit the parent session's permissions; a role description cannot restrict them independently. Enforced read-only work requires appropriate parent settings or a separate environment.
2. **Give them different tasks.** One investigates existing code and reuse; the other checks alternatives, sources or validation risks. Specify goals, inputs, scope and outputs for each. Do not assume they know the chat history or have read project rules.
3. **Let the lead agent reconcile results.** Wait for evidence before dependent edits. For parallel coding, agree on file ownership, interfaces and acceptance criteria. Git worktrees separate working directories, not permissions or shared databases. Assign shared files an owner, configure test environments separately and check after integration.

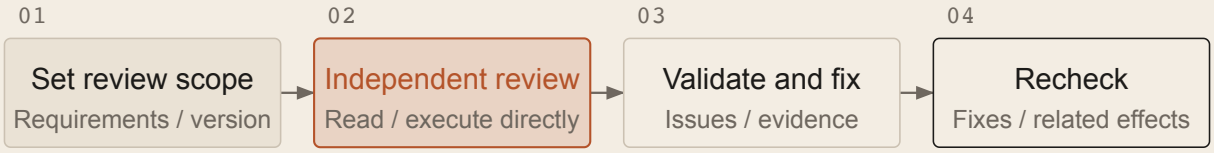
Create Parallel Subtasks

Goal: [task]. Shared materials: [requirements, research, plan and task locations]. Check capabilities and permissions first. If supported, create two read-only subtasks: A checks existing implementation and reuse; B checks [approach, sources or validation risks]. Report missing materials. Subtasks must not edit files or expand scope. If enforced read-only access is required but cannot be configured, pause before launching and explain. Role instructions are not permission isolation. Give each a scope, inputs, checks and return format: findings, locations, evidence and unknowns. The lead agent may advance independent work within current permissions; wait for dependent results and reconcile conflicts first.
Before any parallel editing, establish authorization, file ownership and integration checks. Record running, completed or failed status; no returned result is not completion. If subagents are unsupported, provide task briefs for separate sessions without pretending they launched.

Separate contexts help research, but do not replace project records or guarantee lossless handoffs. Confirm tasks actually started, recover sources and resolve disagreements. Begin small; compare total time, usage and rework before scaling. More agents may increase duplication and coordination cost. Without subagent tools, research in separate chats and consolidate the results.

CHAPTER 9

Get an Independent Agent Review



Verify review findings; agreement among agents is not proof.

Arrange an independent review after important features, integration or before real use. The reviewer should read requirements and current files directly. Relying only on the implementer's completion summary can preserve the same misunderstanding.

Separate implementation from review

1. **Fix the review scope and inputs.** Record the version, changes, requirements and run instructions. Prefer a fresh session or subagent without the implementation conversation, providing only necessary materials. Copying the old chat can import prior judgments.
2. **Report issues by impact.** Give location, trigger, expected and actual behavior, and evidence. Prioritize correctness, security and missing requirements. Keep stylistic preferences optional.
3. **Verify, repair and review again.** The implementer validates each finding; review the current version after fixes. Recheck conclusions affected by changes. Do not add complexity merely to answer every suggestion.

Review Independently and Recheck Fixes

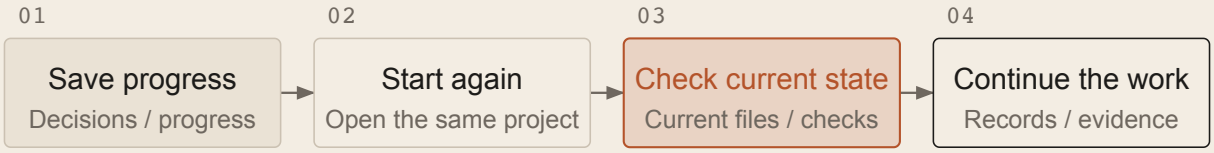
Materials: [original requirements, current requirements, version, change scope and run instructions]. If you helped implement this and tools support it, assign a reviewer who did not participate and does not inherit the implementation conversation. Otherwise provide a handoff brief; you may self-review, but do not call it independent. List missing materials.
Do not edit the implementation under review. Read files first. If execution would write data, use an approved test copy or separate working directory with test data. If writes are prohibited, list checks not run; do not bypass restrictions. Separate directories do not isolate external data.
Check requirements, changes and evidence. Prioritize wrong results, data or permission issues, removed or weakened checks and broken existing behavior. For long-lived projects, also examine obvious duplication and complexity. Give each issue a location, trigger, impact and evidence; distinguish verified defects from risks.
Return a list for the implementer. Hand it off only with authorization and tools; otherwise let the user forward it. After fixes, recheck the current version and summarize resolved, outstanding and unverified items. State when no material issue was found within scope; do not invent findings to meet a quota.

An independent perspective still needs evidence

A new reviewer is less attached to the work, but can make the same mistake. Validate triggers and evidence before repairs. Recheck affected areas and project-required checks afterward. Agreement among several AIs is not proof; ask an experienced person to judge important unresolved questions.

CHAPTER 10

Resume Long Tasks Reliably



History supports resuming; verify what is true now.

Long tasks need clear specs, staged acceptance and persistent records. Review a dependency-ordered queue of specs before execution. Save before interruptions, usage limits or repeated drift; verify the current state before resuming.

Save before stopping; verify before resuming

`PROGRESS.md` is a text progress file; reuse an existing equivalent. Use the first prompt before ending a session and the second after restarting.

Save Progress

Update [progress file]: goals and decisions, project path and version, changed files and uncommitted edits, checks and evidence, failures and unverified items, unfinished work, recovery and next steps. If unable to write, return saveable text without claiming it is on disk.
Index key experiments with hypotheses, conditions, actions, results, abandoned approaches and reasons, and evidence links. Mark replaced decisions; preserve original requirements and evidence. Do not label plans completed. List each spec's status, dependencies and conditions to recheck. Mark missing history unknown rather than inventing it.

Update records after each verifiable stage. On resuming, reread the current spec, progress and relevant experiments. A chat claim of completion does not replace a traceable result.

Read Progress and Continue

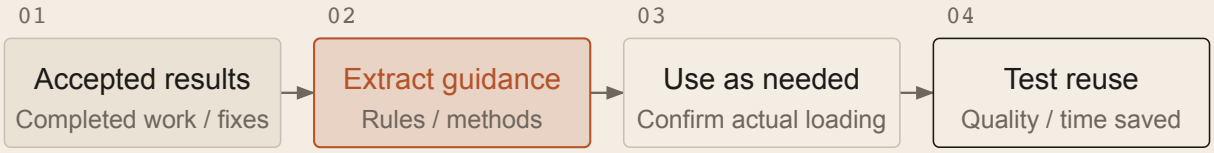
Read [progress file], requirements, plan and tasks. Check the directory, version, existing edits and verification records. Without Git, compare file snapshots. Report missing or unreadable records before guessing progress; do not overwrite others' work.
Continue with the first actionable unfinished task, updating actual results after each verifiable increment. Reuse results only when related code, software, configuration and data are unchanged; recheck if uncertain. Still run the full checks required by the project.
Work within [allowed edit and execution scope] and [budget]. If blocked, advance independent work. Save before the budget runs out; return a handoff if unable to write. Pause affected work when goals must change, permissions must expand or an important unresolved choice needs a decision.

Use acceptance criteria and stop conditions

Within a budget, let AI propose, execute, check and keep useful results without lowering acceptance criteria. Stop when done; save before limits. Pause affected work after repeated failures without new evidence or at unresolved trade-offs. Repetition does not guarantee convergence; restoring code does not undo external actions.

CHAPTER 11

Turn Effective Methods into Rules and Skills



Keep persistent rules brief; load specialized methods as needed.

Capture useful methods when communication or mistakes repeat. Establish safety boundaries from the start; derive other rules from actual problems. More instructions increase reading overhead and do not guarantee accuracy. Test them on a new task.

Keep three kinds of guidance distinct

Form	What it holds and how to use it
Project rules	Special conventions, allowed scope, run and check commands; use recognized files such as `AGENTS.md`, not a full repository overview
Skills	Reusable methods, templates and materials; state when to invoke them, inputs and expected outputs
Automation	Scripts for stable steps; connect a supported Hook when they must run on a specified event

Extract Reusable Rules and Skills

Derive reusable guidance from [accepted results, change records or repeated tasks]. Allowed updates: [file scope]. Reuse project rules. Package repeated methods as a Skill in the current tool's format, and fixed steps as scripts where useful. If Skills or writes are unsupported, return a draft; do not claim activation.

Define applicability, inputs, outputs, essential steps, verification entry points and failure handling. Inspect third-party skills and scripts before use. Preserve valid conventions and avoid unrelated installations.

Report paths and activation steps, including required review, trust, approval or restart. Validate with an authorized independent example; mark unexecuted checks unverified. For automatic checks, verify the trigger and behavior on success, failure and script errors without affecting real data.

Compare quality, communication and rework. Consolidate duplicate or outdated guidance only within scope. Mandatory checks must not depend solely on prompt reminders.

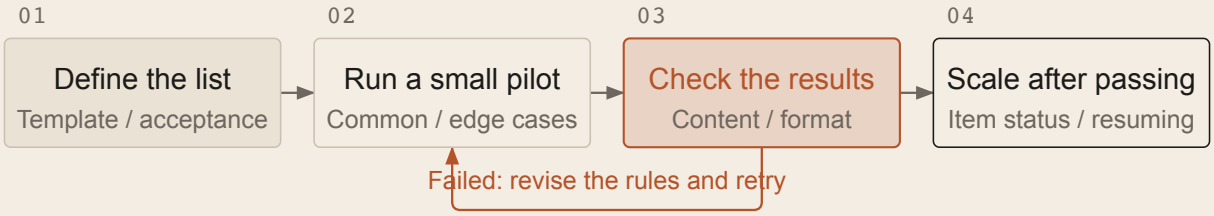
Save standards, not every reasoning step

For example, "Local commits are allowed; pushing to a remote requires authorization" can be a project rule. A repeated research method can become a Skill. Test it on another similar task and remove obsolete guidance.

A configured Hook can call a script on tool-completion events to save commands, results and logs. Background tasks still need their actual exit status checked. A successful command is not acceptance. A logging-only Hook does not block actions. To prevent an action, verify the supported pre-action event and failure behavior in practice; post-action records cannot undo it.

CHAPTER 12

Pilot Repeated Work Before Scaling Up



When many files or tasks need the same treatment, make one-item processing reliable before increasing volume. Batch work repeats the same rules. Parallel execution is appropriate only when items are independent and cannot overwrite one another.

Use a small pilot to establish the batch rules

1. **Define the list and format.** Specify each input, expected output, destination and acceptance criteria. Use a common template and preserve originals.
2. **Start with representative items.** Include common and difficult cases. Check content and format. When AI judgment is involved, repeat a few cases on test copies to assess consistency before scaling.
3. **Record each item and recover unfinished work.** Keep success, failure, skipped or uncertain status with reasons. Recheck affected items after rule, input or environment changes. Pause to fix the process when the same failure recurs.

Pilot and Execute a Batch

Process [task list] using [rules and template]. Output: [location]. Allowed operations and budget: [scope]. Pilot a few common and high-risk cases and show results with evidence. Repeat representative AI judgments to assess consistency. Clarify rules or acceptance criteria first; report missing tools without claiming processing occurred.
Give each item a unique ID, output format, source record and completion criteria. Parallelize only independent items. Do not overwrite originals or repeat external effects. Use copies or safely repeatable operations for repeated trials. Record status, output and failure reason. Check uncertain outcomes before retrying. Resume unfinished items; recheck affected results when inputs, rules or environment change.
Run required checks for every item, then sample actual content. Sampling does not replace required per-item checks. Scale only after the pilot passes and within authorization; pause scaling if a failure pattern repeats. Report the complete list and gaps, total time, human review and rework, not just generation speed.

Fix the process before increasing volume

A small pilot catches errors before they are repeated many times. Review its results and settle the format and criteria first. At scale, reconcile the full list with results and gaps; retain failure reasons. Rule changes require rechecking affected items. A few good outputs do not prove the entire batch is complete.

Twelve Chapters at a Glance

Clarify goals and trade-offs, preserve context in project files, and leave a verifiable result after each increment.

Ch.	Topic	Core idea and action
01	Define an actionable task	**Define outcomes and trade-offs.** Discuss and write the spec; people review scope and observable acceptance criteria.
02	Connect project files	**Keep context in files.** Index requirements, evidence and progress; reread after compaction or a new session.
03	Understand before changing	**Research before architecture.** Compare reuse, adaptation and a new build; verify sources, fit and key unknowns.
04	Use Spec Kit	**Keep requirements connected to execution.** Maintain specs, plans and tasks; checked boxes do not prove completion.
05	Build in small steps	**Make each round verifiable and recoverable.** Save a baseline, connect real behavior and give precise feedback.
06	Check actual use	**Derive expectations from requirements.** Verify full workflows and saved data; separate pass, fail and unverified.
07	Diagnose before editing	**Make failures reusable evidence.** Record hypotheses, conditions and results; keep regression checks after repairs.
08	Split independent work	**Parallelize independent tasks.** Specify inputs and scope; reconcile evidence and measure cost before scaling.
09	Review independently	**Verify review findings too.** Read requirements and the current version; report triggers and evidence; recheck fixes.
10	Resume long tasks	**Advance a queue of specs.** Define dependencies, acceptance and stop conditions; save results and verify before resuming.
11	Reuse rules and Skills	**Keep methods that repeatedly help.** Rules hold conventions, Skills hold methods and scripts run fixed steps; test actual reuse.
12	Pilot, then scale	**Validate the process first.** Pilot common and difficult cases, track every item, resume gaps and measure time and rework.

The thread through all twelve chapters: People own goals, judgment and final acceptance. AI executes within an agreed scope. Files preserve requirements, decisions and experience; evidence shows what is complete. Organize the project before adding automation.

Methods and tools: DORA: small batches; Google SRE: troubleshooting; AWS: architecture decisions; Anthropic: context engineering; OpenAI: subagents; Spec Kit: official workflow. Choose methods for the situation; tool behavior depends on version and configuration.